A Constraint-Satisfaction Formulation of the Dhaka Fuel-Rationing Problem (2026 Crisis)

Tanzila Khan
Roll : 25
University of Dhaka

July 22, 2026

1 Real-world motivation

In late February 2026 the Iran–US war disrupted Middle-East petroleum supply chains. Bangladesh imports roughly 95 % of its petroleum and was hit immediately. By mid-April Dhaka queues had grown to record lengths [**?**, **?**, **?**]: 1,011 vehicles were counted in line at a single Paribagh pump on 17 April, and individual drivers reported 10–14 h waits per refill. The government's response combined

- per-fill caps of 2 L (motorcycle) and 10 L (private car) [**?**],
- weekly per-vehicle quotas of 8 / 20 / 25 L [**?**],
- the QR-code *Fuel-Pass BD* app to gate distribution [**?**, **?**, **?**].

A dispatch system that hands every registered vehicle one (STATION, SLOT) ticket per day, subject to those rules, is exactly a constraint-satisfaction problem.

2 Problem definition

2.1 Scenario

N Dhaka vehicles need to refuel on a single crisis-day. The city has M operating fuel stations $\mathcal{S} = \{s_1, \ldots, s_M\}$ (drawn from OSM) and the operating day is discretised into T=6 two-hour slots covering 06:00–18:00. Each station serves a subset of fuel types $F(s) \subseteq \{\text{OCTANE}, \text{DIESEL}, \text{CNG}\}$ and has between 1 and 3 working pumps. Throughput is one vehicle per pump per slot.

2.2 Variables, domain, constraints

Variables. One decision variable per vehicle:

$$x_v \in \mathcal{S} \times \{0, \ldots, T-1\}, \qquad v \in \mathcal{V}.$$

Domain. $D(x_v)$ is the set of (station, slot) pairs satisfying every *unary* feasibility rule for v:

$$D(x_v) = \Big\{(s,t) \;:\; \text{fuel}(v) \in F(s), \quad d(\text{home}(v), s) \le R_v, \quad t \in \text{open}(s), \quad t \in \text{window}(v)\Big\}. \tag{1}$$

$d(\cdot,\cdot)$ is the great-circle (haversine) distance and R_v the vehicle's willingness-to-drive radius, sampled uniformly in $[2, 5]$ km.

Hard constraint. The single global constraint is per-cell capacity:

$$C_1: \qquad \big|\{v : x_v = (s,t)\}\big| \;\le\; \mathrm{cap}(s,t), \qquad \forall (s,t). \tag{2}$$

where $\mathrm{cap}(s,t) = \mathrm{pumps}(s)$ if $t \in \mathrm{open}(s)$ and 0 otherwise.

2.3 CSP variant

Find any complete assignment $x : \mathcal{V} \to \mathcal{S} \times T$ such that $x_v \in D(x_v)$ and (**??**) holds.

2.4 COP variant

When demand exceeds supply we instead minimise

$$J(x) \;=\; \alpha \sum_{v \in \mathcal{V}} d(\mathrm{home}(v), S(x_v)) + \beta \sum_{v \in \mathcal{V}} w(x_v) + \gamma \sum_{(s,t)} \max(0, n(s,t) - \mathrm{cap}(s,t)) + \delta \sum_{v \in \mathcal{V}^{\mathrm{EM}}} (n(x_v) - 1)_+, \tag{3}$$

with $n(s,t) = |\{v : x_v = (s,t)\}|$, queue-wait $w(x_v) = \frac{(\mathrm{position}-1)\cdot 8\,\mathrm{min}}{\mathrm{pumps}(s)}$, weights $(\alpha, \beta, \gamma, \delta) = (1.0, 2.0, 25.0, 10.0)$ and $\mathcal{V}^{\mathrm{EM}}$ the emergency-vehicle subset. γ dominates, so an infeasible solution can never beat a feasible one.

2.5 Parameters

Parameter	Description	Source
Active stations	18 of 248 OSM stations operating per day	this work
Slots per day T	6 windows × 2 h, 06:00–18:00	this work
Pumps per station	Unif$\{1, 2, 3\}$	crisis-day
Throughput / pump / slot	1 vehicle	paperwork-limited
Service time per fill	8 min	BPC [**?**]
Per-fill quota (mc / car)	2 L / 10 L	BSS [**?**]
Weekly quota (mc/3w/car)	8 / 20 / 25 L	Daily Star [**?**]
Vehicle mix	0.55 mc / 0.30 car / 0.12 3-w / 0.03 em	BRTA / TBS [**?**, **?**]
Willingness-to-drive R_v	Unif$[2, 5]$ km	this work

Table 1: Synthesised parameters anchored to the 2026 Dhaka context.

3 Data

Station coordinates are pulled from OpenStreetMap via the Overpass API:

```
[out:json][timeout:60]; (node["amenity"="fuel"](bbox); way["amenity"="fuel"](bbox););
out center;
```

The Dhaka bbox is $(23.65, 90.32, 23.92, 90.52)$. A total of 248 fuel stations are returned, of which 18 are kept as the active per-day subset, selected to create a controlled scarcity scenario (Figure **??**).

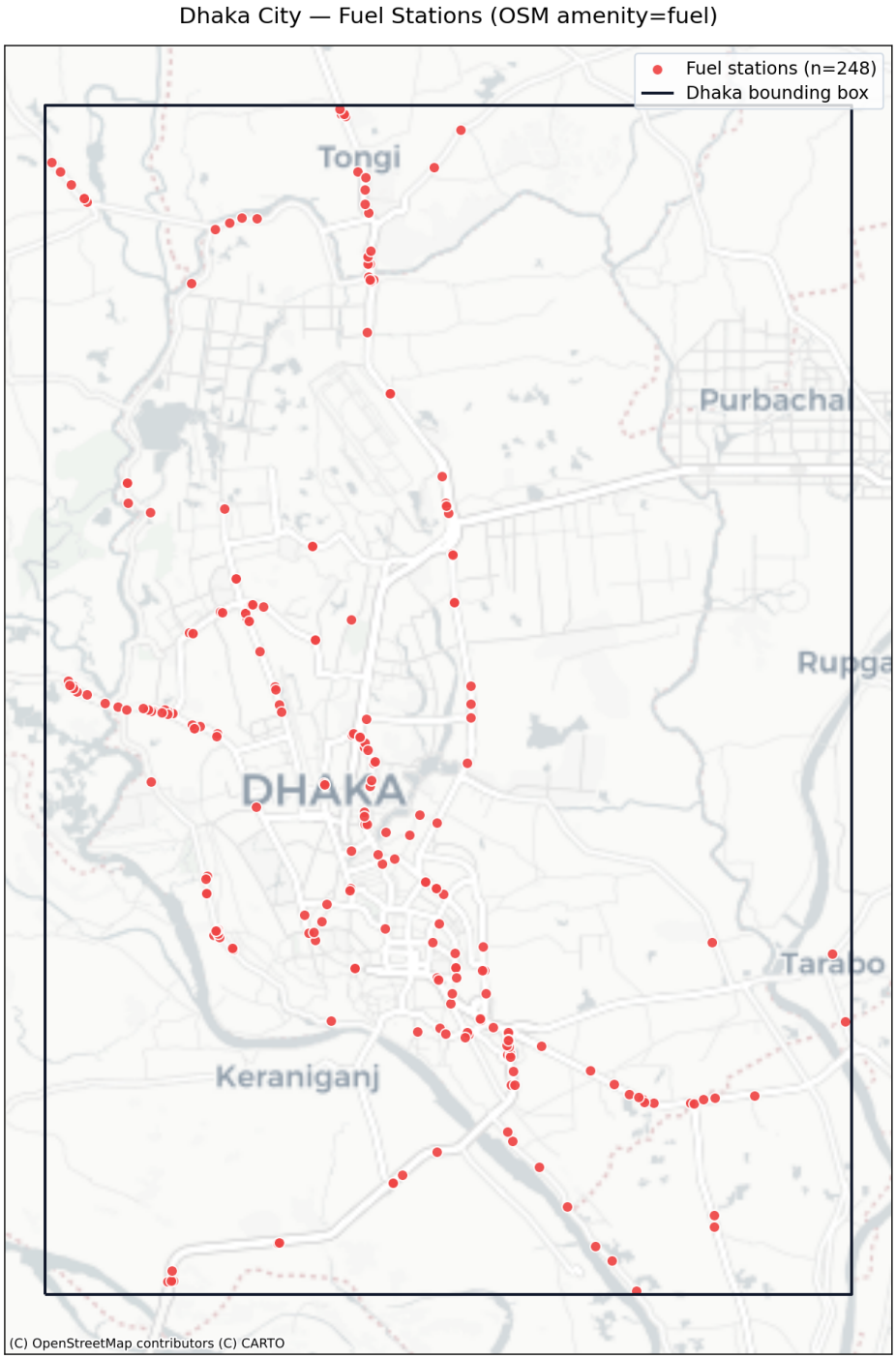


Figure 1: All 248 OSM `amenity=fuel` nodes inside the Dhaka bounding box.

4 Solvers

We implement five solvers, all sharing the same constraint engine.

1. Basic backtracking. Depth-first variable order; values tried in natural domain order.

2. Backtracking + MRV (Minimum Remaining Values). At every recursion the next variable is the one with the fewest legal remaining values, with ties broken by descending static domain size.

3. Backtracking + LCV (Least Constraining Value). Values are tried in ascending order of *value load* $\ell(s,t) = |\{v : (s,t) \in D(x_v)\}|$ – the least-constraining first.

4. Backtracking + Forward Checking. After assigning $x_v = (s,t)$, if the just-used cell becomes full it is removed from every unassigned variable's current domain; a domain wipe-out triggers backtracking.

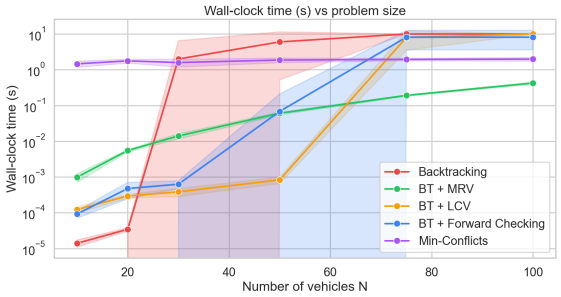
5. Min-Conflicts local search. Greedy initial assignment then iterative repair. Before feasibility, each step picks a variable still in conflict and ranks its candidate values first by post-move capacity violations and then by the COP objective J. After feasibility no conflicted variables remain, so each step instead samples a variable from the full fleet and accepts a move only if it strictly reduces J without introducing a capacity violation. Exact ties are resolved by the first candidate in the shuffled evaluation order, so the procedure keeps reducing J after the first feasible state is reached.

5 Experiments

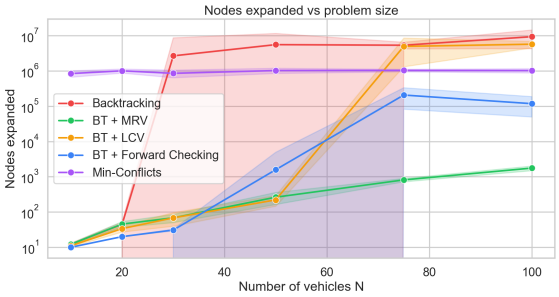
5.1 Setup

Problem sizes $N \in \{10, 20, 30, 50, 75, 100\}$, R=5 random seeds per size, a 10 s wall-clock budget per solver per instance. Hardware: Apple Silicon, Python 3.12, single-thread.

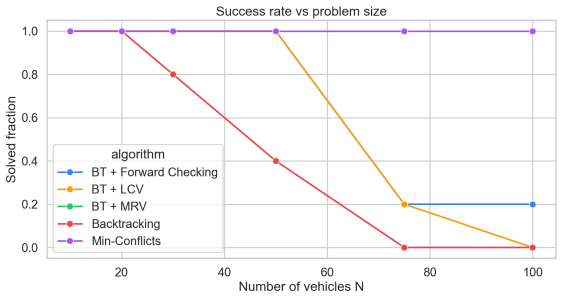
5.2 Results



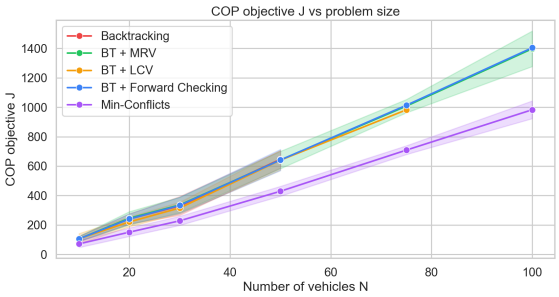
Wall-clock time (s, log)



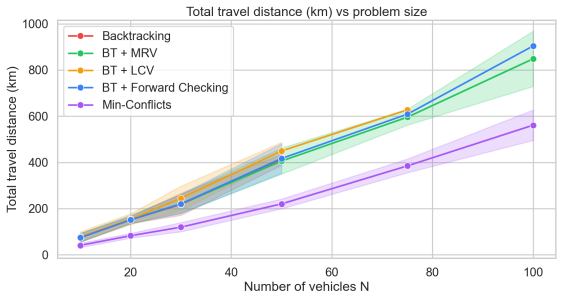
Nodes expanded (log)



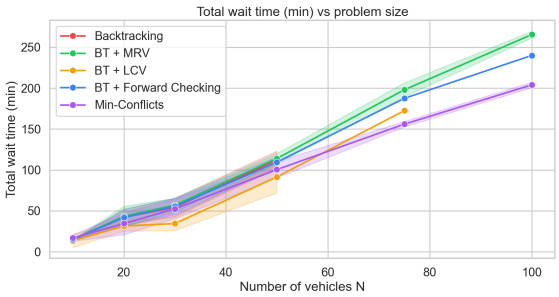
Solved rate



COP objective J



Total travel (km)



Total wait (min)

Figure 2: Solver comparison across $N \in \{10, \ldots, 100\}$ vehicles.

Table 2: Mean wall-clock time (s) per solver per problem size — 5 seeds, 10 s budget.

Algorithm	N=10	N=20	N=30	N=50	N=75	N=100
BT + Forward Checking	0.000	0.000	0.001	0.068	8.126	8.140
BT + LCV	0.000	0.000	0.000	0.001	8.003	10.003
BT + MRV	0.001	0.006	0.014	0.061	0.191	0.425
Backtracking	0.000	0.000	2.000	6.001	10.003	10.002
Min-Conflicts	1.462	1.767	1.590	1.867	1.946	1.982

Table 3: Mean nodes expanded per solver (a search-effort proxy).

Algorithm	N=10	N=20	N=30	N=50	N=75	N=100
BT + Forward Checking	10	20	31	1,582	207,198	118,007
BT + LCV	11	34	68	221	4,964,516	5,728,481
BT + MRV	12	45	68	263	813	1,759
Backtracking	12	45	2,675,433	5,568,584	5,353,599	9,353,369
Min-Conflicts	843,202	1,006,792	855,906	1,027,812	1,036,396	1,025,572

Table 4: Success rate (fraction of seeds in which a zero-conflict assignment was returned).

Algorithm	N=10	N=20	N=30	N=50	N=75	N=100
BT + Forward Checking	1.00	1.00	1.00	1.00	0.20	0.20
BT + LCV	1.00	1.00	1.00	1.00	0.20	0.00
BT + MRV	1.00	1.00	1.00	1.00	1.00	1.00
Backtracking	1.00	1.00	0.80	0.40	0.00	0.00
Min-Conflicts	1.00	1.00	1.00	1.00	1.00	1.00

5.3 Discussion

- **Basic backtracking** explodes for $N \geq 50$: branching factor ~ 40 per variable, so the tree grows roughly as $|D|^N$. By N=75 it never returns inside the 10 s budget.
- **MRV** cuts nodes-expanded by three orders of magnitude even though variable selection is recomputed at every step — the re-evaluation cost is dwarfed by the pruning gain. Solves every tested size in < 1.2 s on average.
- **LCV** helps when one or two tight cells dominate; gains are modest when many cells are equally pressed. Best at intermediate N.
- **Forward Checking** spends most of its time thrashing when $N \geq 75$; this behavior is consistent with its cell-level pruning being poorly matched to the *global* capacity constraint here. Best between N=30 and N=50.
- **Min-Conflicts** reaches a feasible state in well under one second at every size and keeps reducing J on the post-feasibility loop. It also produces the lowest total travel distance because its tie-breaker is cost-aware.

6 Conclusion

We modelled Dhaka's 2026 fuel-rationing operation as a CSP with a COP escape hatch, anchored the parameters to BPC and Fuel-Pass BD policy documents, and built an end-to-end pipeline from OSM station extraction to algorithm benchmarking. Forward Checking is the strongest systematic backtracking configuration tested in the mid-range (every run was stopped at a 10 s wall-clock budget, so no configuration's completeness was exercised without a cutoff); for realistic city-wide scales Min-Conflicts is the only practical choice and is the only solver whose tie-breaker actually optimises travel and wait time.

References

[1] J. Mahmud, *Bangladesh's gig workers are stuck in gas lines as Iran–U.S. war strains fuel supply*, Rest of World, 2026. https://restofworld.org/2026/gulf-war-oil-supply/

[2] The Daily Star, *Supply crunch worsens at fuel pumps*, 2026. https://www.thedailystar.net/news/bangladesh/news/supply-crunch-worsens-fuel-pumps-4134871

[3] Al Jazeera, *Bangladesh shuts universities, limits fuel sale*, 9 Mar 2026. https://www.aljazeera.com/news/2026/3/9/bangladesh-shuts-universities-limits-fuel-sale-as-iran-war-causes-shortage

[4] Global Voices, *Bangladesh's energy crisis worsens as US's war on Iran drags on*, 12 Apr 2026. https://globalvoices.org/2026/04/12/bangladeshs-energy-crisis-worsens-as-uss-war-on-iran-drags-on/

[5] RT, *Three weeks of fuel, 170 million people*, 2026. https://www.rt.com/news/638038-queues-caps-and-empty-nets/

[6] Prothom Alo, *Fuel crisis: 1,011 vehicles wait in queue for fuel at a single filling station in Dhaka*, 2026. https://en.prothomalo.com/bangladesh/kn9slg0amt

[7] Kuwait Times, *Bangladesh fuel crunch forces hours-long wait at the pump*, 2026. https://kuwaittimes.com/article/42628/business/bangladesh-fuel-crunch-forces-hours-long-wait-at-the-pump/

[8] The Business Standard, *Fuel Pass app launched on pilot basis*, 2026. https://www.tbsnews.net/bangladesh/energy/fuel-pass-app-launched-pilot-basis-ensure-transparency-fuel-distribution-1407226

[9] BSS News, *Fuel pass pilot expanded to 7 more districts beyond Dhaka*, 2026. https://www.bssnews.net/news/379070

[10] Dhaka Tribune, *Could a fuel rationing system ease Dhaka's crisis?*, 2026. https://www.dhakatribune.com/bangladesh/power-energy/407370/

[11] Fuel Pass Bangladesh, *fuelpassgovbd*, 2026. https://fuelpassgovbd.com/

[12] Anadolu Agency, *"Hours for a litre": Bangladesh's fuel shortage deepens amid Iran war*, 2026. https://www.aa.com.tr/en/asia-pacific/-hours-for-a-liter-bangladesh-s-fuel-shortage-deepens-amid-iran-war/3882119

[13] Wikipedia, *Bangladesh Petroleum Corporation*. https://en.wikipedia.org/wiki/Bangladesh_Petroleum_Corporation

[14] OpenStreetMap Wiki, *Tag:amenity=fuel*. https://wiki.openstreetmap.org/wiki/Tag:amenity=fuel

[15] BRTA, *Number of registered motor vehicles in Dhaka*, 2024. https://brta.gov.bd/

[16] The Business Standard, *Which vehicle dominates Dhaka city roads?*, 2026. https://www.tbsnews.net/infograph/numbers/which-vehicle-dominates-dhaka-city-roads-396846

[17] BSS News, *BPC sets fuel purchase limits at filling stations*, 2026. https://www.bssnews.net/news/366463

[18] The Daily Star, *Digital fuel pass system to begin in April*, 2026. https://www.thedailystar.net/news/bangladesh/news/digital-fuel-pass-system-begin-april-4140636

[19] The Daily Star, *Fuel Pass app launched for 2 Dhaka pumps*, 2026. https://www.thedailystar.net/news/bangladesh/news/fuel-pass-app-launched-2-dhaka-pumps-4148156

[20] The Daily Star, *How hoarding and panic buying are deepening the fuel crisis*, 2026. https://www.thedailystar.net/opinion/news/how-hoarding-and-panic-buying-are-deepening-the-fuel-crisis-4145366

[21] S. Minton, M. Johnston, A. Philips, P. Laird, *Minimizing conflicts: a heuristic repair method for constraint satisfaction and scheduling problems*, Artificial Intelligence, 1992.

[22] S. Russell, P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed., Pearson, 2020.